Constitutional Agentic Compilation

Bounded-Discretion Realization from Constitutional Specifications

The result is a compilation process in which agents–human or artificial–exercise controlled discretion within a constitutionally defined envelope.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Contemporary software production remains dominated by imperative implementation, where human-authored code is the primary artifact and specifications are secondary, informal, or lossy. This paper introduces **Constitutional Agentic Compilation (CAC)**, a paradigm in which the primary artifact is a constitutional specification–a Codex encoding invariants, configuration space, and admissibility conditions–and executable systems are derived through bounded-discretion transformation.

CAC departs from both classical compilation and unconstrained program synthesis. Unlike traditional compilers, which perform deterministic transformations over fully formalized inputs, CAC operates on semantically structured specifications that deliberately preserve degrees of freedom. Unlike open-ended synthesis, CAC constrains the realization space through explicit invariants, ablation-tested boundaries, and embedded reasoning scaffolds. The result is a compilation process in which agents–human or artificial–exercise controlled discretion within a constitutionally defined envelope.

Correctness in CAC is defined not by output identity but by **invariant preservation and gate-test satisfaction**, yielding equivalence classes of realizations rather than singular outputs. This enables multiple independent agents to produce structurally distinct yet constitutionally equivalent systems from the same specification.

We formalize CAC as a layered transformation pipeline–Human Intent -> Constitutional Codex -> Canonical MCIR -> Agentic Realization -> Executable System–and argue that it represents a new intermediate computational layer between intent and execution. We further show that CAC aligns with governance-oriented frameworks such as semantic hashing, invariant-based verification, and epistemic constraint systems, including Governed Context & Memory (GCM) .

---

1. From Instruction to Constitution

Traditional software systems treat code as the authoritative artifact. Specifications, where present, are advisory: they guide implementation but do not bind it. CAC inverts this relationship.

A **Constitutional Codex** is not a program description in the procedural sense. It does not instruct an executor to "perform operation X." Instead, it defines:

- **Invariants** – properties that must hold across all valid realizations
- **Configuration space** – permissible variation within the system
- **Enforcement structures** – how invariant violations are detected
- **Ablation boundaries** – what the system becomes if a property is removed
- **Verification surfaces** – how satisfaction is externally checkable

The Codex is therefore a **governance artifact**, not a procedural one. It defines the admissible space of systems rather than a single system.

This aligns with domain-level specifications such as the Deployable Software Systems Codex, where invariants define what a system *is*, and ablation defines what it *becomes* when those invariants are removed .

---

2. Compilation Reinterpreted

Classical compilation is defined by three properties:

1. Deterministic transformation
2. No discretion
3. Identical output for identical input

CAC preserves only the first property in a weakened form. The transformation is **deterministic in intent**, but not in representation.

2.1 Bounded Discretion

CAC introduces **bounded discretion**:

- The Codex intentionally leaves implementation space open
- The agent selects within that space
- Choices do not affect invariant satisfaction

Discretion exists in:

- data structures
- algorithm selection
- naming and organization
- internal decomposition

This is not an error surface–it is a designed degree of freedom.

2.2 Equivalence Under Invariants

CAC replaces output identity with **equivalence under invariants**:

> Two realizations are equivalent if they satisfy the same invariant set and pass the same gate tests.

This property–formalized as convergence under independent realization–means:

- Different agents can produce different code
- All valid outputs are members of the same equivalence class
- Structural divergence is defined only by invariant violation

This is consistent with the principle that correctness is externally verifiable via declared properties, not inferred from implementation identity.

---

3. The Agent as Compiler

In CAC, the compiler is redefined as an **agent**.

This agent may be:

- a human developer
- an AI coding system
- a hybrid orchestration system
- a future deterministic lowering engine

What defines the agent is not its implementation, but its role:

> **To produce a realization that satisfies the constitutional constraints of the Codex.**

This aligns with split-authority systems where semantic authority (Governor) and execution authority (Coder) are separated and mediated through a shared artifact.

3.1 Agentic vs Deterministic Compilation

- **Property:** Input
  Traditional Compilation: Fully formal program
  CAC: Constitutional specification
- **Property:** Transformation
  Traditional Compilation: Deterministic
  CAC: Bounded-discretion

- **Property:** Output
  Traditional Compilation: Identical
  CAC: Functionally equivalent
- **Property:** Compiler
  Traditional Compilation: Mechanism
  CAC: Agent
- **Property:** Correctness
  Traditional Compilation: Semantic equivalence
  CAC: Invariant satisfaction

3.2 Human and AI Equivalence

CAC treats humans and AI as interchangeable realizers:

- both interpret invariants
- both select implementations
- both are verified by the same gate tests

However, AI agents expose constraint gaps more reliably, as they do not implicitly repair underspecification.

---

4. The CAC Transformation Pipeline

CAC introduces a multi-stage transformation model:

```
Human Intent
  v (Crystallization)
Constitutional Codex (HLIR)
  v (Canonicalization)
MCIR (Canonical, hash-stable representation)
  v (Agentic Compilation)
Executable Realization (source code or IR)
  v
Runtime Execution
  v
Effects + Verification Receipts
```

4.1 HLIR -> MCIR

The Codex is transformed into a canonical intermediate representation:

- governance-relevant structure preserved
- representation normalized

- identity becomes hash-stable

This parallels semantic hashing systems, where canonicalization defines equivalence classes of meaning .

4.2 MCIR -> Realization

This is the **agentic compilation step**:

- constraints enforced
- configuration respected
- implementation space filled

Today, this step is performed by AI agents. In the future, it may become deterministic.

4.3 Toward CEIR

The long-term trajectory introduces a lower-level intermediate representation:

- CEIR (Constitutional Execution IR)
- directly executable
- derived deterministically from MCIR

At this point, CAC converges with classical compilation.

---

5. Verification as First-Class Structure

CAC externalizes correctness.

Rather than relying on compiler guarantees, the Codex defines:

- **gate tests**
- **violation conditions**
- **observable properties**

This aligns with the requirement that valid systems must expose a **verification surface**–a structural link between declared behavior and observable reality .

Verification is therefore:

- explicit
- testable
- independent of implementation

---

6. Relation to Existing Paradigms

CAC intersects with several known paradigms but is not reducible to any:

- **Paradigm:** Program synthesis
  Overlap: Generates code from constraints
  Limitation: Lacks structured invariants
- **Paradigm:** Model-driven engineering
  Overlap: Models -> code
  Limitation: Models lack ablation and governance
- **Paradigm:** Declarative systems
  Overlap: Specify "what" not "how"
  Limitation: Limited reasoning scaffolds
- **Paradigm:** Formal methods
  Overlap: Verifiable correctness
  Limitation: Often impractical at scale

CAC combines:

- declarative constraint definition
- structural reasoning scaffolds
- verifiable realization
- bounded synthesis

---

7. Governance and Epistemic Integrity

CAC is not only a computational model–it is a governance model.

It shares structural alignment with systems such as GCM, where:

- meaning is externalized
- invariants are enforced
- drift is detectable
- ignorance is admissible

In such systems:

$$\text{Unknown} > \text{Speculation} > \text{Fabrication}$$

Similarly, CAC enforces:

- no silent ambiguity
- no implicit reinterpretation
- no unverifiable claims

Failures become explicit and diagnosable.

---

8. Properties and Advantages

8.1 Reduced Search

Agents operate in a constrained space:

- fewer retries
- lower compute cost
- minimal thrash

8.2 Executor Independence

Multiple agents can:

- produce valid realizations
- without coordination
- from the same Codex

8.3 Regenerability

Systems can be:

- rebuilt from specification
- transformed across targets
- audited at the semantic level

8.4 Cold-Start Capability

Because reasoning is embedded:

- agents require no prior context
- Codex acts as a complete reasoning scaffold

---

9. Failure Modes

CAC introduces distinct failure modes:

9.1 Underspecified Codex

- agent fills gaps incorrectly
- system remains internally consistent

9.2 Incorrect Invariants

- system is valid but misaligned with reality

9.3 Boundary Leakage

- S/C separation violated
- configuration mistaken for invariant

These failures are structural and must be surfaced through:

- ablation testing
- cross-validation
- independent realization

---

10. Toward Deterministic Constitutional Compilation

The ultimate form of CAC removes agentic discretion from the execution phase:

```
MCIR -> CEIR -> Execution
```

At this point:

- compilation becomes deterministic
- reproducibility becomes exact
- CAC becomes a new class of declarative programming

The Codex becomes the program.

---

11. Conclusion

Constitutional Agentic Compilation defines a new computational layer in which:

- systems are governed before they are implemented
- reasoning is encoded before execution
- correctness is defined structurally rather than procedurally

It replaces:

> instruction -> execution

with:

> constitution -> admissible realization

In CAC, the compiler is no longer a passive translator. It is an agent operating within a governed semantic field.

And the program is no longer code.

It is the structure that makes code inevitable.

Standing and Claim Boundary

This paper is a conceptual, formal, or architectural contribution unless it explicitly binds a statement to a cited external source or an exact carried evidence artifact. Unbound statements about implementation, validation, performance, safety, universality, or proof are research claims or design obligations, not established operational facts. Corpus-internal references establish conceptual lineage and comparison, not independent external confirmation.

Realization Scope and Evidence Boundary

This successor makes the paper's stage boundary explicit. It narrows claims rather than manufacturing missing evidence: a stage named as context is not thereby claimed as implemented, validated, or complete.

- **Instance Configuration.** The present paper does not claim a complete instance configuration treatment. That stage remains successor work and cannot be inferred from the architecture described here.
- **Revision.** The present paper does not claim a complete revision treatment. That stage remains successor work and cannot be inferred from the architecture described here.

Corpus-Wide Reference Positioning

This paper argues that contemporary software production remains dominated by imperative implementation, where human-authored code is the primary artifact and specifications are secondary, informal, or lossy. Its nearest external comparison fields are formal, governance, open sovereign. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. A written constitution can legitimate domination as easily as constrain it unless amendment,

participation, enforcement, and appeal remain effective in practice. Source availability and reproducible builds improve inspectability but do not alone guarantee comprehensibility, practical exit, hardware control, or sustainable maintenance. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] George C. Necula. "Proof-Carrying Code." *POPL '97*, 1997, pp. 106-119. DOI: 10.1145/263699.263712. https://doi.org/10.1145/263699.263712 **Relationship:** Predecessor for binding machine-checkable evidence to executable artifacts; the present paper may demand broader semantic or constitutional obligations.

[**E2**] Xavier Leroy. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52(7), 2009, pp. 107-115. DOI: 10.1145/1538788.1538814. https://doi.org/10.1145/1538788.1538814 **Relationship:** Predecessor verified-compilation result used to distinguish proven translation preservation from broader governance claims.

[**E3**] Patrick Cousot and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *POPL '77*, 1977. DOI: 10.1145/512950.512973. https://doi.org/10.1145/512950.512973 **Relationship:** Predecessor formal framework for sound approximation and fixpoint reasoning.

[**E4**] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. https://lamport.azurewebsites.net/tla/book.html **Relationship:** Predecessor specification discipline for state machines, safety, liveness, and refinement.

[**E5**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763 **Relationship:** Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E6**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/
Relationship: Adjacent legal-philosophical account of limited authority; included to expose differences between political and computational constitutionalism.

[**E7**] Lawrence Lessig. *Code: Version 2.0*. Basic Books, 2006. https://codev2.cc/
Relationship: Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

Ableman corpus

[**A1**] Adam Ableman. *MCIR: A Canonical Intermediate Representation for Governed Transformation*. Ableman Corpus current edition; work identity `PAPER.PUB.21095303`;
source SHA-256
`EA129ECBA95F7B07CAA7B33A54F91F52913C94A14DA227EB9A06CFD083E4E5BF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Governed Semantic Compilation*. Ableman Corpus current edition; work identity `PAPER.PUB.18453681`;
source SHA-256
`5FF6C636550B04DB0E2F6F94C379BB75AA0BD8781E19B047D7D3BB6D49071AC2`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Hybrid Semantic Compiler Architectures*. Ableman Corpus current edition; work identity `PAPER.PUB.21228038`;
source SHA-256
`EBCD6DDDA9B547639AEBED79FB68069473CF1248E41C527B48C9624BBE2C0496`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Admissibility Engineering*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-F1E33B5E973B0900`;
source SHA-256
`CDC80271268802CA5CB4DE3624B992FB2FF07D677BF945C4B349BD307C4C2A6E`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Everything Is a Compiler*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-30FB8F95BD990A47`;
source SHA-256
`FE0F88A1F125317285BE4378E5F27ACC9FEB83ADE3FF329A3DA9F9CB026DF9A9`.

Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Ableman Constitutional Governance.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-79353AF163A4B0F1`;
source SHA-256
`FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Constitutional Theory.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-D2D97C4042FD281F`;
source SHA-256
`FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Adam Ableman *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Zenodo, 2026. doi:10.5281/zenodo.21095303

Adam Ableman *Semantic Coordinate Identity Tokenization (SCIT).* Zenodo, 2026. doi:10.5281/zenodo.20532070

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848574